

Program:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

void findfirst(char, int, int);
void followfirst(char, int, int);
void follow(char);

int count, n = 0, m = 0;
char calc_first[10][100];
char calc_follow[10][100];
char production[10][10];
char first[10], f[10];
int k;
char ck;
int e;

int main() {
    int i;
    char c;
    int num;

    printf("Enter number of productions: ");
    scanf("%d", &count);

    printf("\nEnter the productions in the form A=BC (use # for epsilon):\n");
    for (i = 0; i < count; i++) {
        printf("Production %d: ", i + 1);
        scanf("%s", production[i]);
    }

    int kay;
    char done[count];
    int ptr = -1;

    // Initialize FIRST array
    for (k = 0; k < count; k++) {
        for (kay = 0; kay < 100; kay++) {
            calc_first[k][kay] = '!';
        }
    }

    printf("\n----- FIRST SETS ----- \n");
```

```
int point1 = 0;
int point2, xxx;

for (k = 0; k < count; k++) {
    c = production[k][0];
    point2 = 0;
    xxx = 0;

    // Check if FIRST already calculated
    for (kay = 0; kay <= ptr; kay++)
        if (c == done[kay])
            xxx = 1;

    if (xxx == 1)
        continue;

    n = 0;
    findfirst(c, 0, 0);
    ptr += 1;
    done[ptr] = c;

    printf("First(%c) = { ", c);
    calc_first[point1][point2++] = c;

    for (i = 0; i < n; i++) {
        int lark, chk = 0;
        for (lark = 0; lark < point2; lark++) {
            if (first[i] == calc_first[point1][lark]) {
                chk = 1;
                break;
            }
        }
        if (chk == 0) {
            printf("%c ", first[i]);
            calc_first[point1][point2++] = first[i];
        }
    }
    printf("}\n");
    point1++;
}

// FOLLOW computation
char donee[count];
ptr = -1;
for (k = 0; k < count; k++) {
```

```
    for (kay = 0; kay < 100; kay++) {
        calc_follow[k][kay] = '!';
    }
}

printf("\n----- FOLLOW SETS ----- \n");

point1 = 0;
for (e = 0; e < count; e++) {
    ck = production[e][0];
    point2 = 0;
    xxx = 0;

    for (kay = 0; kay <= ptr; kay++)
        if (ck == donee[kay])
            xxx = 1;
    if (xxx == 1)
        continue;

    m = 0;
    follow(ck);
    ptr += 1;
    donee[ptr] = ck;

    printf("Follow(%c) = { ", ck);
    calc_follow[point1][point2++] = ck;

    for (i = 0; i < m; i++) {
        int lark, chk = 0;
        for (lark = 0; lark < point2; lark++)
            if (f[i] == calc_follow[point1][lark])
                chk = 1;
        if (chk == 0) {
            printf("%c ", f[i]);
            calc_follow[point1][point2++] = f[i];
        }
    }
    printf("}\n");
    point1++;
}

return 0;
}
```

// FOLLOW set calculation

```
void follow(char c) {
    int i, j;
    if (production[0][0] == c)
        f[m++] = '$'; // Start symbol gets $

    for (i = 0; i < count; i++) {
        for (j = 2; j < strlen(production[i]); j++) {
            if (production[i][j] == c) {
                if (production[i][j + 1] != '\0')
                    followfirst(production[i][j + 1], i, j + 2);
                if (production[i][j + 1] == '\0' && c != production[i][0])
                    follow(production[i][0]);
            }
        }
    }
}
```

// FIRST set calculation

```
void findfirst(char c, int q1, int q2) {
    int j;
    if (!(isupper(c))) {
        first[n++] = c;
        return;
    }

    for (j = 0; j < count; j++) {
        if (production[j][0] == c) {
            if (production[j][2] == '#') {
                if (production[q1][q2] != '\0')
                    findfirst(production[q1][q2], q1, q2 + 1);
            } else
                first[n++] = '#';
        } else if (!(isupper(production[j][2]))) {
            first[n++] = production[j][2];
        } else {
            findfirst(production[j][2], j, 3);
        }
    }
}
```

// FOLLOW-FIRST helper

```
void followfirst(char c, int c1, int c2) {
    int k;
    if (!(isupper(c))) {
```

```

    f[m++] = c;
} else {
    int i = 0, j = 1;
    for (i = 0; i < count; i++)
        if (calc_first[i][0] == c)
            break;

    while (calc_first[i][j] != '!') {
        if (calc_first[i][j] != '#')
            f[m++] = calc_first[i][j];
        else {
            if (production[c1][c2] == '\0')
                follow(production[c1][0]);
            else
                followfirst(production[c1][c2], c1, c2 + 1);
        }
        j++;
    }
}
}
}

```

Output:

```

/home/student/22co16/expt8
Enter number of productions: 8
Enter the productions in the form A=BC (use # for epsilon):
Production 1: E=TR
Production 2: R=+TR
Production 3: R=#
Production 4: T=FY
Production 5: Y=*FY
Production 6: Y=#
Production 7: F=(E)
Production 8: F=id

----- FIRST SETS -----
First(E) = { ( i }
First(R) = { + ( i }
First(T) = { ( i }
First(Y) = { * ( i }
First(F) = { ( i }

----- FOLLOW SETS -----
Follow(E) = { $ ) }
Follow(R) = { $ ) }
Follow(T) = { + ( i }
Follow(Y) = { + ( i }
Follow(F) = { * ( i }

Process returned 0 (0x0)   execution time : 67.112 s
Press ENTER to continue.

```

Conclusion: Program to calculate the FIRST and FOLLOW sets of a grammar via user input was implemented and executed successfully.